TOBB ETÜ

Ekonomi ve Teknoloji Üniversitesi

Delta Design & Implementation Report

Color Clash - A Two-Player Fighting Game

TOBB ETU – BIL 482

April 2026

Team Members

Mehmet Umur ÖZÜ

Mehmet Yasin TOSUN

Mustafa GÖZÜTOK

1. Introduction

This document presents the delta-oriented evaluation of the Color Clash project after the review phase. Its purpose is to record the review feedback, identify what can be stated from repository-visible evidence, formalize the related quality factors, and connect the remaining open points to testing and final delivery artifacts.

The content of this report is intentionally limited to what is visible in the submitted review report, project repository, and final delivery artifacts. It does not claim that optional recommendations were implemented unless such implementation is directly supported by the available evidence.

2. Baseline System Under Review

The review report describes Color Clash as a web-based 2D fighting game developed with TypeScript, HTML5 Canvas, and Vite. The reviewed system supports local multiplayer and a tournament mode, and its core gameplay is built around elemental modes and a combat system.

The same review also states that the architecture integrates the Singleton, Factory, State, Strategy, Command, Observer, and Object Pool patterns. It identifies GameEngine, InputHandler, CollisionSystem, and Character as the key architectural roles, and notes that the game loop starts reliably from init().

The repository structure is consistent with this description. The project is organized into source modules such as core, entities, patterns, systems, and UI, supported by documentation and a dedicated test folder.

3. Review Feedback Summary

Feedback ID	Review Finding	Classification
RF-01	Input keys are hard-coded and do not allow user-defined control schemes.	Weakness
RF-02	Use Case 4 was incomplete at review time.	Weakness / Missing mechanism
RF-03	Asynchronous asset loading is minimal and may limit future scalability.	Missing mechanism
RF-04	Collision detection edge cases should be tested rigorously.	Recommendation
RF-05	Performance benchmarks should be verified.	Condition for stronger acceptance
RF-06	Configuration API and FPS counter are recommended refinements.	Improvement recommendation
RF-07	AI Opponent is technically feasible but optional.	Optional future improvement
RF-08	Networking should be avoided for the MVP due to synchronization complexity.	Explicit non-selection

3.1 Review Feedback Resolution Mapping

Feedback ID	Review Finding	Final Team Position	Evidence Basis	Final Status
RF-01	Hard-coded controls	Accepted as an MVP limitation; default controls remain usable for local play.	Final scope interpretation and usability-oriented local gameplay validation	Open limitation
RF-02	Use Case 4 incomplete	The final project delivery includes completed tournament flow and final validation coverage for tournament progression and final podium presentation.	Final test coverage and delivery alignment	Resolved
RF-03	Asynchronous asset loading minimal	No unsupported implementation claim is made; the item is preserved as future improvement scope.	Delta scope statement	Deferred
RF-04	Collision edge cases need rigorous testing	Targeted collision-related validation was included, but exhaustive edge-case coverage is not claimed.	Final automated validation scope	Partially addressed
RF-05	Performance benchmarks should be verified	Stable execution is supported qualitatively by build, smoke, and manual validation, but no formal benchmark study is claimed.	Final test interpretation	Open validation limitation
RF-06	Configuration API and FPS counter recommended	Not implemented and not claimed in the final scope.	Delta scope statement	Deferred
RF-07	AI Opponent optional	Treated as future work rather than final MVP scope.	Review interpretation	Deferred
RF-08	Avoid networking in MVP	Preserved as a deliberate non-selection to protect scope and stability.	MVP scope definition	Confirmed non-selection

4. Selection of Delta Scope

Given the available evidence, the most defensible delta scope is not a speculative feature-expansion narrative. Instead, the appropriate scope is documentation and validation centered: clarifying the architecture, formalizing quality factors, strengthening testing traceability, and recording pending items from the review honestly.

- Architecture visibility and traceability
- Quality-factor formalization
- Testing and bug-fix documentation
- Explicit recording of pending review items

By contrast, items such as configurable key remapping, a dedicated Configuration API, an FPS counter, advanced asynchronous loading, AI opponent support, or networking should not be claimed as implemented unless separate evidence is provided.

5. Delta Assessment and Design Impact

The review already evaluates the project positively. It explicitly highlights pragmatic pattern usage, modularity, type safety, and real-time game-loop performance. As a result, the delta discussion should be framed as consolidation rather than re-architecture.

Accordingly, the most accurate design-impact statement is that the existing architecture was preserved and clarified. The repository-visible structure continues to show a clean separation between core orchestration, game entities, pattern abstractions, gameplay systems, and user-interface components.

6. Architecture Update Statement

No repository evidence currently supports a claim that the top-level architecture was replaced or that a new design-pattern family was introduced after review. The correct statement for the final report is therefore that the delta phase maintained the existing architecture and improved its explanation, traceability, and validation framing.

- Core: central orchestration and game execution flow
- Entities: player and game-domain objects
- Patterns: reusable behavioral and structural abstractions
- Systems: gameplay-related processing
- UI: player-facing interface and interaction flow

6.1 Test and Documentation Impact of the Delta Phase

The delta phase did not introduce a new architectural family or a top-level redesign. Instead, its practical impact was concentrated in three areas. First, architectural intent became more visible through clearer documentation and traceability framing. Second, quality factors were formalized as explicit evaluation artifacts. Third, final validation evidence became stronger through requirement-linked testing, manual scenario coverage, and regression-oriented smoke execution. Therefore, the delta contribution should be interpreted as consolidation and evidence strengthening rather than structural redesign.

7. Quality Factor Table

The following table is the quality-factor artifact requested for the final deliverables. It is based on repository-visible capabilities, review-visible risks, and final delivery evidence.

Quality Factor	Quality Criteria	Metric	Target Value	Related Review Item	Related Evidence / Test Artifact
Functionality	Local multiplayer works correctly	Two players can play on the same keyboard in one match	Supported in final build	Baseline method and scope	Final demonstration scope and functional validation
Functionality	Tournament mode exists and progresses sequentially	Tournament flow available with player registration and elimination structure	Present in documented scope	Baseline scope	Final test coverage for tournament flow and podium output
Modularity	Source code is decomposed into clear subsystems	Presence of dedicated architectural folders	core / entities / patterns / systems / ui preserved	Strength: modularity	Repository structure inspection
Maintainability	Pattern-based responsibilities remain explicit	Pattern layer exists and is separated from subsystems	Dedicated patterns directory maintained	Strength: pragmatic design	Repository structure inspection
Reliability	Smoke-level validation is executable	Existence of automated smoke command	npm run test:smoke available	Validation strengthening	package.json + automated test execution
Usability	Controls are understandable for local play	Default control scheme is usable even if not configurable	Acceptable for MVP; customization missing	RF-01	Manual validation and accepted limitation statement
Performance	Real-time execution remains smooth	Browser-loop based rendering architecture	Qualitatively acceptable; no formal benchmark claimed	RF-05	Build verification, smoke validation, and manual gameplay validation
Scalability	Asset loading can support future growth	Asynchronous loading mechanism maturity	Currently limited / pending improvement	RF-03	Review finding and future-work note

Quality Factor	Quality Criteria	Metric	Target Value	Related Review Item	Related Evidence / Test Artifact
Testability	Critical interaction paths can be validated	Collision, input, and scenario flow can be tested	Targeted collision validation included	RF-04	Final test report and targeted automated checks

7.1 Interpretation of Quality Factor Coverage

The quality-factor table should be read together with the final test report. Functionality is supported by validation of match setup, controls, winner/result handling, and tournament progression. Reliability is supported by the completed test cycle and smoke-test execution. Testability is strengthened by requirement-linked automated and manual tests, while performance remains acceptable at the qualitative level but is not claimed as formally benchmarked.

8. Testing and Bug-Fixing Discussion

The review set a clear validation agenda for the project: Use Case 4 should be completed, collision detection should be tested rigorously, and performance benchmarks should be verified. These points were carried into the final delivery documents as traceable validation responsibilities.

At the same time, the repository already included a smoke-test command, which showed that a baseline automated validation artifact existed. The correct delta interpretation is therefore that the project contains an initial automated test layer, while deeper scenario-specific and performance-oriented validation is now clarified through the final test report.

8.1 Fixed Bugs / Open Issue Record

Record ID	Description	Source	Current Status
B-01	Hard-coded input bindings reduce flexibility and prevent user-defined remapping.	Review finding	Open / accepted MVP limitation
B-02	Use Case 4 was incomplete at review time.	Review finding	Completed and validated within final project delivery scope
B-03	Collision edge cases require stronger validation.	Review recommendation	Partially addressed through targeted automated collision validation; exhaustive edge-case coverage not claimed
B-04	Performance benchmarks should be verified explicitly.	Review condition	Formal benchmark still pending; qualitative stability supported by build, smoke, and manual validation

8.2 Test Configuration Note

Field	Value
Environment	Local browser-based execution environment
Build Tool	Vite
Main Language	TypeScript
Available Automated Validation	Smoke-test command in project configuration
Further Validation Need	Formal performance benchmark evidence remains outside the current final claim

9. Prioritization Discussion

The review itself suggests the correct prioritization strategy for the remaining duration. Mandatory scope completion and validation depth should come before optional expansion. This means that completing unfinished required functionality and strengthening collision/performance verification is more appropriate than introducing networking or other risky extensions.

This prioritization also aligns with the review's explicit statement that networking should be avoided for the MVP, while optional additions such as an AI opponent belong to future phases rather than the essential final delivery.

9.1 Final Delta Position

The final delta position of Color Clash is that the project did not rely on speculative expansion after review. Instead, it strengthened final deliverables through clearer architectural explanation, improved traceability, a formal quality-factor artifact, and final-cycle validation evidence. In this sense, the delta phase improved project defensibility for demonstration and submission without overstating unverified implementation claims.

10. Conclusion

Color Clash already had a strong architectural baseline at review time. The review positively evaluated its design-pattern usage, modularity, type safety, and performance-oriented game-loop structure. The repository also confirms a modular source layout, dedicated documentation, and an existing smoke-test entry point.

The main delta need is therefore not to invent unverified implementation claims, but to improve the traceability of the final deliverables. In this context, the Delta Design & Implementation Report and the Quality Factor Table serve as the artifacts that connect the review findings, architectural interpretation, and validation expectations in a clear and honest form.